

A Faithful Lean Account of the PZG–BEDC Kernel

Unified Theory Working Line

Abstract

We report the portion of the PZG–BEDC dynamic ontology kernel that is presently formalized in Lean 4 with mathlib. PZG denotes prime-axis Zeckendorf coding: each prime axis carries a finite Zeckendorf word, and decoding first turns those words into prime exponents and then reconstructs a positive natural number. The checked development covers the finite generation layer, a Newman normal-form criterion, prime-axis factorization and Euclid escape, Zeckendorf and PZG bijections, normalization as multiplication, finite golden-ratio double-sided algebra, integrality of the normalization deficit, finite reading and a prime two-square classifier, Cantor diagonal barriers for self-coding, and a cost-based time arrow for event histories. Every mathematical assertion labeled as machine checked below names an actual Lean declaration under `papers/unified_theory/lean4/UnifiedTheory`; no theorem claim is made for the analytic zeta program, quasicrystal zeta, zero transport, RH, or any other item not represented by a real Lean declaration. The Lean project is mathlib-based and isolated from the mathlib-free BEDC core; the stated reproducibility surface is the branch `feat/unified-theory-mathlib`, with the full project check given by `lake build`.

1 Introduction

The source theory proposes a dynamic mathematical ontology kernel

$$\mathcal{K}_{\text{PZG}}$$

whose objects are not only static data, but generated histories, standard codes, decoders, normalizers, finite readings, and ledger states. The PZG part of the name is literal: *prime axes* provide the coordinate directions, while *Zeckendorf words* provide the per-axis digit system. In the formalized carrier, a PZG table is a finite-support map

$$p \longmapsto w_p$$

where the support is restricted to primes and each w_p is a legal Zeckendorf representation. Decoding maps each w_p to a natural exponent and then uses unique factorization to recover an element of $\mathbb{N}_{>0}$.

The goal of this Lean line is not to certify the full speculative reach of the source document. It is narrower and more valuable as a logical audit: identify the finite and algebraic claims that can be stated cleanly, prove them in Lean, and keep analytic or self-referential frontier claims outside the checked-result column until they have genuine declarations and proofs. This paper therefore distinguishes three classes.

- (i) **Checked theorem or definition:** a declaration in `UnifiedTheory/**.lean` that can be referenced by name.
- (ii) **Encoded statement:** a Lean `Prop` used to record a target without pretending that it is proved.
- (iii) **Outlook:** mathematical plans or analytic programs that are not yet formalized.

The development is deliberately mathlib-based. It uses mathlib’s APIs for natural-number factorization, Zeckendorf representations, golden-ratio identities, pigeonhole arguments, and Fermat’s theorem on sums of two squares. This is separate from the mathlib-free BEDC core in the main repository: no file under the BEDC core imports this Lean project, and this project does not alter the BEDC no-mathlib invariant.

2 Formalized Results

This section lists the checked content by layer. Each item gives an informal statement, the precise Lean declaration name, and the reason it captures the corresponding part of the source theory.

2.1 Generation, Mark Exchange, and Histories

Two marks and exchange. The finite basis of the generation layer is the inductive type

`UnifiedTheory.Mark.`

It has exactly two constructors, `zero` and `one`. The exchange operation is

`UnifiedTheory.Mark.sigma : Mark -> Mark,`

with checked facts

`UnifiedTheory.Mark.sigma_involutive`

and

`UnifiedTheory.Mark.sigma_nontrivial.`

Informally, the first theorem says $\sigma(\sigma(x)) = x$ for both marks, and the second records that σ really swaps the two constructors. This is the formal core of the source definitions 1.1–1.2: the minimal binary marker and its nontrivial exchange are not axioms, but a two-constructor inductive type and a case split.

Finite histories. The history carrier is

`UnifiedTheory.MarkHist,`

with constructors `empty` and `ext`. The formal append operation satisfies

`UnifiedTheory.MarkHist.empty_append,`
`UnifiedTheory.MarkHist.append_empty,`
`UnifiedTheory.MarkHist.append_assoc,`

and its length satisfies

`UnifiedTheory.MarkHist.length_append.`

These are the checked version of the finite-history monoid laws in proposition 2.4. The source document describes generation induction and event finiteness as axiomatic principles; the Lean line does not add such axioms. It realizes the finite layer by ordinary inductive recursion and lists.

Events and generation. Events are structures

`UnifiedTheory.Event`

with fields for source history, opcode, argument, and output tag. Event histories are lists, and generation is list extension:

`UnifiedTheory.EventHist.generate.`

The checked statement

`UnifiedTheory.EventHist.generate_length`

says that appending one event increases the list length by one. This is the finite operational shadow of the source idea that a generated event extends a history.

2.2 Unicity by Abstract Rewriting

The formal rewriting module is generic in a type α and a relation $r : \alpha \rightarrow \alpha \rightarrow \text{Prop}$. It defines the reflexive-transitive closure

`UnifiedTheory.Rewriting.Star,`

normal forms

`UnifiedTheory.Rewriting.Normal,`

joinability, local confluence, confluence, and termination:

`UnifiedTheory.Rewriting.Joinable,`

`UnifiedTheory.Rewriting.LocallyConfluent,`

`UnifiedTheory.Rewriting.Confluent,`

`UnifiedTheory.Rewriting.Terminates.`

The checked Newman theorem is

`UnifiedTheory.Rewriting.newman_confluent.`

Its type is:

$\text{Terminates}(r) \rightarrow \text{LocallyConfluent}(r) \rightarrow \text{Confluent}(r).$

The normal-form theorem is

`UnifiedTheory.Rewriting.exists_unique_normal_form,`

which states that under the same hypotheses every element has a unique normal form. This exactly formalizes the source theorem 3.2 at the abstract rewriting level. It does not assert that every future normalizer in the theory has already been given a terminating rewrite system; it supplies the reusable criterion.

2.3 Prime Axes and Euclid Escape

The prime-axis state space is

`UnifiedTheory.PrimeExp,`

a finite-support natural exponent vector whose support consists of primes. The declaration

`UnifiedTheory.PrimeExp.equivPNat`

has type

$\mathbb{N}_{>0} \simeq \text{UnifiedTheory.PrimeExp}.$

This is the formalized prime-axis statement: positive natural numbers are equivalent to finite prime exponent vectors. The proof is a mathlib wrapper around `Nat.factorizationEquiv`, so the paper should not claim that this Lean file rederives the full elementary proof of unique factorization from scratch.

The exponent reading is

`UnifiedTheory.vp,`

and the multiplicative additivity theorem is

`UnifiedTheory.vp_mul.`

It says

$$v_p(mn) = v_p(m) + v_p(n)$$

for $m, n \in \mathbb{N}_{>0}$. This is the checked content behind theorem 4.4’s “multiplication is coordinate-wise addition on prime axes” slogan.

Euclid escape is represented by

`UnifiedTheory.euclidEscape`

and proved by

`UnifiedTheory.exists_prime_not_mem.`

For every finite set S of primes, the theorem produces a prime divisor of

$$\prod_{p \in S} p + 1$$

that is not in S . This is the formalized version of theorem 4.5: no finite window of prime axes exhausts the prime directions.

2.4 Zeckendorf Words and the PZG Bijection

On a single axis, the Zeckendorf carrier is

`UnifiedTheory.AxisWord,`

a subtype of lists satisfying mathlib’s `List.IsZeckendorfRep`. The equivalence

`UnifiedTheory.AxisWord.equivNat`

has type

$$\mathbb{N} \simeq \text{UnifiedTheory.AxisWord.}$$

The round-trip theorems

`UnifiedTheory.AxisWord.decode_encode,` `UnifiedTheory.AxisWord.encode_decode`

state the existence and uniqueness directions of Zeckendorf representation. This is the checked theorem 5.3, again using mathlib’s Zeckendorf API rather than a handwritten proof.

The PZG table carrier is

`UnifiedTheory.PZGTable.`

It is not merely an exponent vector: it keeps the per-prime Zeckendorf word. The axis-level equivalence

`UnifiedTheory.PZGTable.equivPrimeExp`

decodes each axis word to an exponent and preserves prime support. Composing this with prime factorization gives

`UnifiedTheory.PZGTable.decode : PZGTable  $\rightarrow$  simeq  $\rightarrow$  Npos .`

The checked round trips

`UnifiedTheory.PZGTable.decode_encode, UnifiedTheory.PZGTable.encode_decode`

are the formal statement of theorem 5.5: PZG tables and positive natural numbers are in bijection.

Normalization is defined canonically:

`UnifiedTheory.PZGTable.normAdd z w = encode (decode z * decode w).`

The theorem

`UnifiedTheory.PZGTable.decode_normAdd`

says

$$\text{decode}(\text{normAdd}(z, w)) = \text{decode}(z) \text{ decode}(w),$$

and

`UnifiedTheory.PZGTable.vp_decode_normAdd`

says the same thing coordinatewise as prime-exponent addition. This captures theorem 5.6 as a canonical normalization theorem. It is not a proof of a step-by-step carry algorithm or carry termination; it defines the normalized result by the uniqueness of the PZG bijection.

2.5 Golden Double-Sided Algebra

The golden algebra carrier is

`UnifiedTheory.PhiInt.`

An element is an integer pair representing $a + b\varphi$ with $\varphi^2 = \varphi + 1$. Multiplication, conjugation, and real embeddings are defined by

`UnifiedTheory.PhiInt.conj,`
`UnifiedTheory.PhiInt.toRealPlus,`
`UnifiedTheory.PhiInt.toRealMinus.`

The theorem

`UnifiedTheory.PhiInt.fixed_conj_iff`

says that a `PhiInt` is fixed by conjugation exactly when its φ -coefficient is zero. This is the algebraic reason the deficit integrality proof works.

The embedding compatibility theorem

`UnifiedTheory.PhiInt.toRealMinus_eq_toRealPlus_conj`

identifies the shrinking real face with the expanding real face after conjugation, while

`UnifiedTheory.PhiInt.toRealPlus_mul`

states that the expanding embedding is multiplicative. The Fibonacci decomposition of powers is

`UnifiedTheory.PhiInt.phiPow_ab,`

which says that the b -coordinate of φ^n is the Fibonacci number F_n .

For axis words, the double-sided reading is

`UnifiedTheory.AxisWord.betaPhi.`

The checked gap identity

`UnifiedTheory.AxisWord.betaPhi_gap`

says that the difference between the expanding and shrinking real readings is $\sqrt{5}$ times the ordinary decoded value. At the PZG-table level, the two real readings are

`UnifiedTheory.PZGTable.lambdaPlus,` `UnifiedTheory.PZGTable.lambdaMinus.`

These declarations formalize definition 6.4 as a finite sum over the finite support of a PZG table.

Carry ledger identities. The finite golden carry identities are checked as

`UnifiedTheory.Golden.gold_carry_plus,` `UnifiedTheory.Golden.gold_carry_minus,`

and the bottom-rule identities as

`UnifiedTheory.Golden.gold_bottom_plus,` `UnifiedTheory.Golden.gold_bottom_minus.`

The exact `PhiInt` version is

`UnifiedTheory.Golden.phiPow_carry.`

These are the formal finite algebra behind proposition 6.21: adjacent Fibonacci carries preserve the double-sided account, while the bottom identity records the unit deficit equation $2\varphi^2 - \varphi^3 = 1$.

Deficit integrality. The normalization deficit is defined by

`UnifiedTheory.deficitPhi.`

The theorem

`UnifiedTheory.deficitPhi_integer`

states that for all $v, w : \mathbb{N}$ there exists $k : \mathbb{Z}$ such that

$$\text{deficitPhi}(v, w) = \text{offInt}(k).$$

The equivalent conjugation-fixed statement is

`UnifiedTheory.deficitPhi_conj_fixed.`

This is the checked theorem 6.22: the deficit is an integer because its φ -coordinate cancels. The stronger three-value statement is not a theorem in the current Lean development; see Section 3.

Finite Fibonacci core. The finite recurrence

`UnifiedTheory.Golden.finiteW_recurrence`

and Cassini identity

`UnifiedTheory.Golden.cassini`

formalize the finite algebraic core behind the later trace-map and generating-function discussion. They do not formalize the infinite function $W(x, y)$, the quasicrystal zeta object, Euler products, analytic continuation, or zero transport.

2.6 Finite Readings and Two-Square Classification

The finite-reading pigeonhole theorem is

`UnifiedTheory.Reading.reading_has_fiber.`

Given a finite carrier, a finite class set, and a map into that class set, if the class set has smaller cardinality than the carrier, two distinct elements share the same reading. This proves theorem 8.5 at the level of finite sets: a finite reading with too few values has a nontrivial fiber.

The two-square classifier currently formalized is the prime-level theorem

`UnifiedTheory.Reading.prime_sq_add_sq_iff.`

For a prime p , it states

$$(\exists a b : \mathbb{N}, a^2 + b^2 = p) \iff p \bmod 4 \neq 3.$$

The easy obstruction is proved in the file from the fact that squares modulo 4 are 0 or 1, and the existence direction uses mathlib’s theorem for primes that are sums of two squares. This captures the prime-classifier part of chapter 9. It is not the full theorem for arbitrary n with all 3 mod 4 prime exponents even.

2.7 Self-Coding and Diagonal Barriers

The self-code layer presently contains Cantor diagonal theorems for arbitrary types. The declaration

`UnifiedTheory.SelfCode.no_classifier_surjective`

says that no classifier

$$X \rightarrow \text{Set}(X)$$

is surjective. The declaration

`UnifiedTheory.SelfCode.diagonal_not_classified`

exhibits the diagonal property $\{y \mid y \notin \text{classify}(y)\}$ and proves it is not hit by any index. The dual declaration

`UnifiedTheory.SelfCode.no_decoder_injective`

says that no decoder $\text{Set}(X) \rightarrow X$ is injective.

These are theorem 16.2–16.3 in their pure diagonal form. They do not formalize the full program-evaluation fixed-point machinery, the same-layer closure predicate, or the internalization tower described in the source theory.

2.8 Dynamics and the Cost Arrow

For event histories, the checked cost theorem is

`UnifiedTheory.EventHist.cost_lt_generate.`

It states that generating one event strictly increases the length of the event history. The theorem

`UnifiedTheory.EventHist.generate_ne`

then says that the generated history is not equal to the original history.

This is the formal content of theorem 18.11–18.12 currently present in Lean: irreversibility is expressed as strict growth of the event ledger. It is not yet the full prime-address dynamics with logarithmic cost over signed exponent states.

2.9 Kernel Components and Ledger States

The kernel aggregate is intentionally thin. The status enum

```
UnifiedTheory.Kernel.LedgerStatus
```

has four constructors:

```
open, closed, tail, semantic.
```

The component module supplies aliases such as

```
UnifiedTheory.Kernel.History, UnifiedTheory.Kernel.ExponentState,  
UnifiedTheory.Kernel.CodeRegister, UnifiedTheory.Kernel.decode, UnifiedTheory.Kernel.normalize.
```

These names stabilize the currently formalized slots of $\mathcal{K}_{\text{PZG}}$. They do not constitute a theorem that every component of the full source-theory kernel has been built as one closed record.

3 Scope and Honesty

The following boundaries are part of the result.

The deficit trichotomy is statement-only. The Lean declaration

```
UnifiedTheory.DeficitTrichotomy
```

is a `Prop`, not a theorem. It states the desired conclusion that the integer deficit lies in $\{-1, 0, 1\}$, but no Lean proof of this declaration is present. The checked part is only:

```
UnifiedTheory.deficitPhi_integer,  
UnifiedTheory.deficitPhi_conj_fixed.
```

Thus theorem 6.25 is not counted among the machine-checked theorems here. The missing ingredient is the global shrinking-face window bound for legal Zeckendorf words.

Normalization is canonical, not algorithmic carry termination. The theorem

```
UnifiedTheory.PZGTable.decode_normAdd
```

proves that the canonical normalized table decodes to the product. The current Lean code does not prove termination of a concrete carry-rewrite procedure for Zeckendorf tables.

The analytic layer is not yet formalized. The infinite generating function $W(x, y)$, the quasicrystal zeta object Z_{qc} , Euler-product factorizations beyond the finite algebraic carriers, meromorphic continuation, natural-boundary claims, zero transport, completed zeta, theta/Mellin machinery, spectral geometry, and RH are not represented by checked Lean theorems in this project. They belong to outlook unless and until they receive real theorem declarations and proofs.

Chapter 9 is represented only at prime level. The declaration

```
UnifiedTheory.Reading.prime_sq_add_sq_iff
```

classifies when a prime is a sum of two squares. The broader classical theorem for arbitrary n in terms of even exponents at primes $q \equiv 3 \pmod{4}$ is not a current Lean theorem in this project.

The self-code layer is diagonal, not a full evaluator. The checked self-code theorems are Cantor diagonal facts about maps $X \rightarrow \text{Set}(X)$ and $\text{Set}(X) \rightarrow X$. They do not yet supply a verified instruction set, substitution evaluator, Kleene fixed-point theorem, or same-layer closure-decider contradiction inside the full kernel machine.

No RH or zero-location claim is proved. The source theory contains a rich ledger language for projection zeros, local accounts, mirror pairing, positivity candidates, and several faces of RH. None of that is a checked theorem in `UnifiedTheory/**`. This paper therefore makes no theorem claim about zeta zeros.

4 Outlook / Future Work

All items in this section are planned or exploratory and are **not yet formalized**.

Analytic zeta program. The intended analytic line is to connect prime-axis freedom, PZG heat traces, Euler products, completed zeta, and finite-window tail budgets. The current Lean project contains the prime-axis and PZG bijections, but not the analytic continuation or completed zeta machinery.

Quasicrystal zeta and diagonal feedback. The source theory proposes a quasicrystal zeta object Z_{qc} built from the golden double-sided reading, with a diagonal feedback problem labeled O-5. The present Lean project contains only finite golden algebra: `PhiInt`, λ_+/λ_- definitions, carry identities, deficit integrality, and Cassini. The analytic Z_{qc} program is not yet formalized.

RH and its three faces. The theory source discusses RH through local ledger conditions, reflection geometry, and spectral or positivity readings. These are not Lean theorems in this project. A faithful Lean route would first need completed zeta, the relevant analytic function theory, a precise positivity bridge, and a clear separation between projection zeros and any stronger ontology-level zero notion.

Internalization tower. The present diagonal module gives Cantor-style unreachability of classifiers. A later internalization tower would require verified syntax, coding, substitution, evaluator semantics, layer-indexed ledgers, and relative diagonal theorems. None of those are currently checked beyond the pure diagonal facts listed above.

Automated science as correspondence search. The intended research workflow is to search for structural overlaps between the PZG–BEDC kernel and known external mathematics: Euler products with prime exponent vectors, Beatty–Wythoff and Zeckendorf coding, $\mathbb{Q}(i)$ two-square norms with $\mathbb{Q}(\sqrt{5})$ deficit traces, Hecke–Mahler series with golden word factors, and so on. This is a promising organizational program, but in the present Lean line it is only supported by the finite correspondences actually listed in Section 2.

5 Reproducibility

The checked source lives under

```
papers/unified_theory/lean4/UnifiedTheory.
```

The working branch is

```
feat/unified-theory-mathlib.
```

The Lean project is pinned by

```
papers/unified_theory/lean4/lean-toolchain
```

and

```
papers/unified_theory/lean4/lakefile.lean.
```

It is a mathlib project, separate from the mathlib-free BEDC core. The full verification command is:

```
cd papers/unified_theory/lean4 && lake build.
```

The branch described here is intended to have that check green, with no `sorry` and no `axiom` in the `UnifiedTheory` sources.

For individual claims, the smallest reproducibility surface is Lean’s type checker. For example:

```
##check UnifiedTheory.Rewriting.exists_unique_normal_form
##check UnifiedTheory.PZGTable.decode
##check UnifiedTheory.PZGTable.decode_normAdd
##check UnifiedTheory.deficitPhi_integer
##check UnifiedTheory.Reading.prime_sq_add_sq_iff
##check UnifiedTheory.SelfCode.no_classifier_surjective
```

Each checked theorem named in this manuscript has been chosen to correspond to an existing declaration, not to a similarly named item in the source prose.